Otimização de Consultas

*Prof. Me. **Orlando** da Silva Junior*

Do quê estamos **falando**?



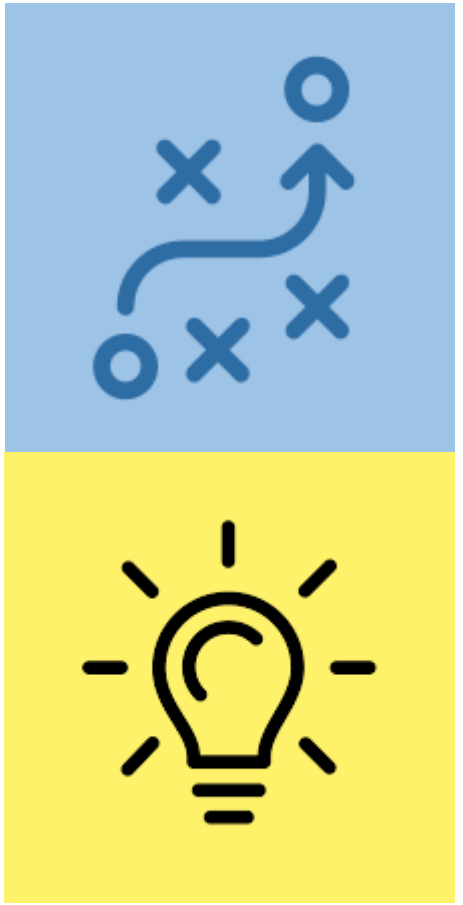
- Uma consulta expressa em SQL primeiro precisa ser **lida**, **analisada** e **validada**:
  - A **varredura** identifica os tokens de consulta (palavras-chave SQL, nomes de atributo e nomes de relação) que aparecem no texto da consulta.
  - O **analisador sintático** verifica a sintaxe da consulta para determinar se ela está formulada de acordo com as regras de sintaxe (gramática da linguagem).
  - A consulta também precisa ser **validada** verificando se todos os nomes de atributo e relação são válidos e semanticamente significativos no esquema do BD.

- A consulta é, em geral, realizada como uma **árvore de consulta**, que é uma estrutura de dados que representa internamente a consulta.
  - Também é possível representar a consulta por meio de um **grafo de consulta**.

Estratégias

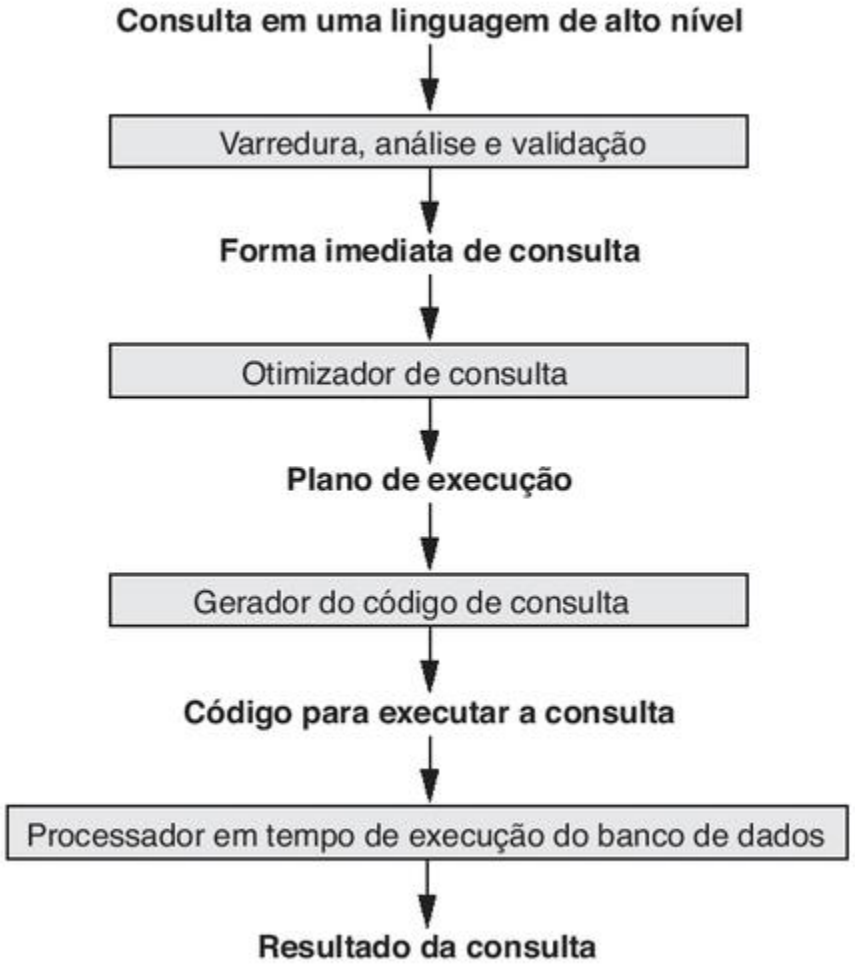
- O SGBD precisa idealizar uma **estratégia de execução** (ou **plano de consulta**) para recuperar os resultados da consulta com base nos arquivos de BD.
  - Uma consulta pode ter várias estratégias de execução.

- Escolher a melhor estratégia é um processo chamado **otimização de consulta**.



Como **otimizar** uma consulta?

- Para otimizar uma consulta, precisamos conhecer como ela é executada. A figura ao lado mostra as diferentes etapas do processo de otimização de consultas.
- O módulo **otimizador de consulta** tem a tarefa de produzir um bom plano de execução;
- O **gerador de código** dá origem ao código para executar esse plano.
- O **processador em tempo de execução do BD** executa o código da consulta, sejam no modo compilado ou interpretado.
  - Se houver um erro em tempo de execução, uma mensagem de erro é gerada pelo processador.



Etapas típicas do processamento de consultas
(ELMASRI & NAVATHE, 2011)

Mas o que significa **otimizar**?



- Apesar de ser o termo mais comum, *otimização* é um nome errado para descrever esse processo.
  - Em alguns casos, o plano de execução escolhido não é a estratégia ótima, mas apenas uma estratégia razoavelmente eficiente.
  - Encontrar a melhor estratégia em geral é muito demorado.

- O ***planejamento de uma boa estratégia de execução*** pode ser uma mais precisa do que devemos fazer.

- Porque SQL é uma **linguagem declarativa** (ou seja, descrevemos o conjuntos das tarefas que resolvem o problema, e não o passo-a-passo algorítmico da solução), não precisamos escrever a estratégia de execução enquanto escrevemos o banco de dados (como acontece em SGBDs de rede ou hierárquicos).

Otimização em **SGBDs relacionais**

- Um SGBDR deve avaliar sistematicamente estratégias alternativas de execução de consulta e escolher uma estratégia quase ideal;

- Em geral, os SGBDs têm uma série de algoritmos gerais de acesso de banco de dados que implementam operações da álgebra relacional, como seleção e junção.

- **Somente** estratégias de execução que podem ser implementadas pelos algoritmos de acesso ao SGBD e que se aplicam à consulta podem ser consideradas pelo módulo de otimização de consultas.

Tradução de consultas SQL para álgebra

- Para realizar a otimização, o SGDB traduz a consulta SQL para uma expressão equivalente da álgebra relacional estendida, que é representada como uma estrutura de dados de árvore de consulta.
  - Após isso, a otimização é realizada.



- As consultas são **decompostas** em **blocos de consulta**:
  - Formam as unidades básicas que podem ser traduzidas em operadores algébricos e otimizadas;
  - Contêm uma única expressão SELECT-FROM-WHERE e cláusulas GROUP BY/HAVING;
  - Operadores de agregação (SUM, MIN, COUNT, etc.) também são incluídos.

Objetivo: recuperar os nomes dos funcionários de qualquer departamento na empresa que ganham um salário maior que o maior salário no departamento 5.

```
SELECT  Unome, Pnome
FROM    FUNCIONARIO
WHERE   Salario > ( SELECT  MAX (Salario)
                    FROM    FUNCIONARIO
                    WHERE   Dnr=5 );
```

Essa consulta possui uma subconsulta aninhada, que seria decomposta na otimização como:

```
( SELECT  MAX (Salario)
  FROM    FUNCIONARIO
  WHERE   Dnr=5 )
```

$$\Im_{\text{MAX Salario}}(\sigma_{\text{Dnr}=5}(\text{FUNCIONARIO}))$$

No bloco de consulta mais externo, *c* representa o resultado retornado do bloco interno:

```
SELECT  Unome, Pnome
FROM    FUNCIONARIO
WHERE   Salario > c
```

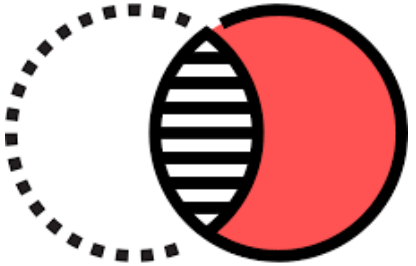
$$\pi_{\text{Unome,Pnome}}(\sigma_{\text{Salario}>c}(\text{FUNCIONARIO}))$$

Algoritmos para **ordenação externa**

- A **ordenação** é um dos principais algoritmos usados no processamento de consultas.
  - Sempre que uma consulta SQL especifica um ORDER BY, o resultado precisa ser ordenado;
  - É um componente-chave da junção, união, intersecção e projeção (DISTINCT);

- A **ordenação externa** é adequada para arquivos grandes de registros armazenados em disco que não se ajustam inteiramente na memória principal.
  - Exige espaço de buffer (cache do SGBD)!

- A principal estratégia de ordenação externa é a **sort-merge**:
  1. Inicia classificando pequenos subarquivos (pedaços) do arquivo principal;
  2. Mescla os pedaços e cria subarquivos maiores;
  3. Intercala os subarquivos maiores.



Como o **SELECT** é implementado?

- A **operação de seleção** é basicamente uma operação de pesquisa para localizar os registros em um arquivo de disco.
  - Essa operação satisfaz algumas condições.

- Existem duas estratégias de pesquisa para selecionar os arquivos de um registro:
  1. **Varredura de arquivo (*file scan*):** esta estratégia varre os registros de um arquivo para procurar e recuperar os registros que satisfazem a condição de seleção;
  2. **Varredura de índice (*index scan*):** o algoritmo envolve o uso de um índice.



Algoritmos de **pesquisa**

- Exemplos de **algoritmos de pesquisa** que podem implementar a operação de seleção:
  1. **Pesquisa linear:** recupera cada registro no arquivo e testa se seus valores de atributo satisfazem a condição de seleção;

  2. **Pesquisa binária:** utilizada em arquivos ordenados para recuperar valores quando a condição de seleção envolver uma comparação de igualdade em um atributo chave;

  3. **Usando índice primário:** usada quando a condição de seleção envolve uma comparação de igualdade em um atributo chave com um índice primário.
     - Exemplo: CPF = ‘12345678910’
     - **Índice primário para vários registros:** utilizando operadores $>$, $\geq$, $<$ ou $\leq$;

Mais algoritmos de **pesquisa**

- Mais alguns exemplos de **algoritmos de pesquisa**:
  4. **Usando uma chave hash:** usada quando a condição de seleção envolve uma comparação de igualdade em um atributo chave com uma chave hash;

  5. **Usando índice de agrupamento:** usada para comparações de igualdade em atributos não chave com um índice de agrupamento.
     - Exemplo: FUNCIONARIOS.Id_departamento = 5

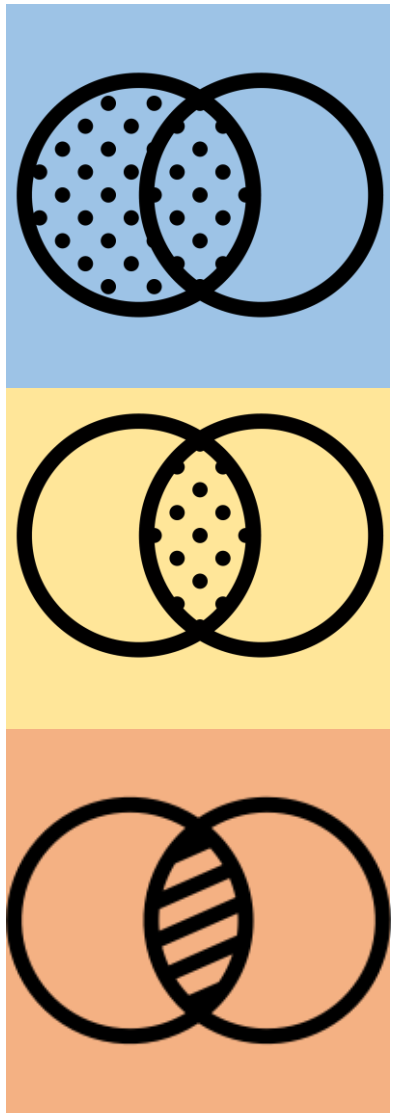
  6. **Usando índice secundário (B+-tree) em uma comparação de igualdade:** pode ser utilizado para recuperar um único registro se o campo de índice for uma chave ou para recuperar múltiplos registros se o campo de índice não for uma chave.
     - Para múltiplos registros, utilizar comparações envolvendo $>$, $\geq$, $<$ ou $\leq$;

Como o **JOIN** é implementado?

- A **operação de junção** é uma das operações mais demoradas no processamento da consulta.

- Junções que são realizadas em:
  - Dois arquivos são chamadas de **junção de duas vias**;
  - Mais de dois arquivos são chamadas de junções **multivias**.

- O número de vias possíveis para implementar cresce muito rapidamente!



Algoritmos para **junções**

- Existem alguns **métodos** para implementar junções:
  1. **Junção de loop aninhado:** para cada registro t em R (loop externo), recupere cada registro s de S (loop interno) e teste se os dois registros satisfazem a condição de junção $t[A] = s[B]$;

  2. **Junção de único loop:** se houver um índice (ou chave hash) para um dos dois atributos de junção, recupere cada registro t em R e depois use a estrutura de acesso para recuperar diretamente todos os registros correspondentes s de S que satisfazem $s[B] = t[A]$;

Junção **sort-merge**

- Se os registros de R e S estiverem fisicamente ordenados por valor dos atributos de junção A e B, respectivamente, podemos implementar a junção da maneira mais eficiente possível:
  - Os dois arquivos são varridos simultaneamente na ordem dos atributos de junção, combinando os registros que têm os mesmos valores para A e B.

- Nesse método, pares de blocos de arquivos são copiados para buffers de memória na ordem e os registros de cada arquivos são varridos apenas uma vez cada, para combinar com o outro arquivo.

1. Ordenação das tuplas;
2. Leitura simultânea dos arquivos R e S pelos atributos A e B;
3. Valida as condições;
4. Realiza a intercalação.

```
ordena as tuplas em R sobre atributo A;          (* suponha que R tem n tuplas (registros) *)
ordena as tuplas em S sobre atributo B;          (* suponha que S tem m tuplas (registros) *)
atribua i ← 1, j ← 1;
enquanto (i ≤ n) and (j ≤ m)
faça {   se R(i)[A] > S(j)[B]
                então atribua j ← j + 1
         se não se R(i)[A] < S(j)[B]
                então atribua i ← i + 1
         se não {   (* R(i)[A] = S(j)[B], de modo que enviamos uma tupla combinada *)
                envia a tupla combinada <R(i), S(j)> para T;

                (* envia outras tuplas que combinam com R(i), se houver *)
                atribua l ← j + 1;
                enquanto (l ≤ m) and (R(i)[A] = S(l)[B])
                faça { envia a tupla combinada <R(i), S(l)> para T;
                       atribua l ← l + 1
                }
                (* envia outras tuplas que combinam com S(j), se houver *)
                atribua k ← i + 1;
                enquanto (k ≤ n) and (R(k)[A] = S(j)[B])
                faça {   envia a tupla combinada <R(k), S(j)> para T;
                         atribua k ← k + 1
                }
                atribua i ← k, j ← l
         }
}
```

Algoritmo de junção ordenação-intercalação (sort-merge) (ELMASRI & NAVATHE, 2011)

Junção sort-merge / **Análise**

- O algoritmo é muito eficiente se os dois arquivos já estiverem ordenados por seu atributo de junção.
  - Somente um único passo é feito em cada arquivo.
  - Assim, o número de blocos acessados é igual à soma dos números de blocos nos dois arquivos.

Se considerarmos qualquer uma das seguintes operações:

$$\text{FUNCIONARIO} \bowtie_{\text{Dnr=Dnumero}} \text{DEPARTAMENTO}$$

$$\text{DEPARTAMENTO} \bowtie_{\text{Cpf_ger=Cpf}} \text{FUNCIONARIO}$$

Podemos <u>estimar</u> o custo total de uma junção sort-merge por:

$$(\boldsymbol{b_F + b_D + b_F \log_2 b_F + b_D \log_2 b_D})$$

b_F = número total de blocos lidos para o arquivo de loop externo

b_D = número de blocos de disco

Algoritmos para implementar a **Projeção** (π)

- A **operação de <u>projeção</u>** é:
  - **Simples** de se implementar, se a lista de atributos incluir a chave da relação R;
  - **Difícil**, se a lista não incluir uma chave de R, pois as tuplas duplicadas deverão ser eliminadas.

```
    cria uma tupla t[<lista atributos>] em T' para cada tupla t em R;
            (* T' contém os resultados da projeção antes da eliminação de duplicatas *)
    se <lista atributos> inclui uma chave de R
            então T ← T'
    se não   {   ordena as tuplas em T' ;
                 atribuai ← 1, j ← 2;
                 enquanto i ≤ n
                 faça {   envia a tupla T' [ i ]  para T;
                          enquanto T' [ i ]  = T' [ j ]  e j ≤ n do j ← j + 1;    (* elimina duplicatas *)
                          i ← j; j ← i + 1
                 }
}
(* T contém o resultado da projeção após a eliminação de tuplas *)
```

Implementação da operação de projeção. (ELMASRI & NAVATHE, 2011)

E na **prática**?

- Acesse nossa sala no Google Classroom;
- Baixe o arquivo *BD_SQL_Exercicios.docx*;
- Implemente os exercícios utilizando o SQL Server.

